CrossTalk: Channel-Scoped Coordination for Heterogeneous AI Coding Agents

Josh Ayokhai (@ajokhai) October 2026 github.com/ajokhai/cross-talk

ABSTRACT

Developers now run several AI coding agents on one repository at once, often from different vendors: Claude Code in one terminal, Gemini CLI or Codex in another, a script or a local model elsewhere. Each agent is driven by its own harness, and none of them can see the others. They overwrite each other's edits, duplicate work, and promise things nobody can check. We present CrossTalk, an open-source hub that gives such agents a shared, private conversation (a *channel*) with four primitives: messages delivered once, leased advisory file locks, *claims* that turn "on it" into a promise with a heartbeat, and *attention* tracking that tells "connected" apart from "reading". Agents join over MCP, remote MCP, WebSocket, HTTP long-polling, a CLI, or a serial and radio bridge, so the same channel can hold a frontier model, a shell script and a microcontroller. We identify the *turn-boundary problem*: agents built on request/response tool protocols cannot be woken by a new message, so a connected agent may never read it. We describe the mechanisms CrossTalk uses to make that failure rare and, when it happens, visible. In a concurrent-edit benchmark, uncoordinated agents lost 59% of their edits on average in the most contended setting; with CrossTalk locks, 0 of 3,500 edits were lost across 30 runs, at a median lock round trip of 0.3 ms. Messages reached all 49 other members of a 50-member channel with a median latency of 3.3 ms, and a silent claim surfaced less than 5 s after its deadline.

1 Introduction

A single coding agent works in a loop: read files, reason, edit, run tests. Two such agents on the same working tree interfere in ways a single agent never sees. One reads `src/auth.ts`, spends a minute deciding what to change, and writes the file back without the edit the other agent made in that minute. One says it will write the tests, then its session ends, and the other waits. A third is still connected but stopped reading the channel ten minutes ago, so a question addressed to it goes nowhere.

These failures are familiar from distributed systems: lost updates, liveness, and failure detection. What is new is that the participants are language-model agents run by different products. They cannot share memory, they speak through whatever tool interface their

harness provides, and most of them only act when a human or a tool result gives them a turn. Studies of multi-agent LLM systems attribute a large share of failures to coordination between agents rather than to individual capability [12].

The common workarounds each have a cost. Sharing context (pasting transcripts, keeping a running summary, a notes file every agent re-reads) makes every agent pay for everything shared, on every update. Separate git worktrees avoid edit collisions but defer the conflict to merge time, when the agents that understood the change may be gone. Orchestration frameworks [4, 5, 6] coordinate agents they spawn themselves, inside one program; they do not help when the agents are independent products the developer already uses.

CrossTalk takes a smaller position. It does not plan, decompose or route tasks. It provides a place where independently run agents can find each other, say short things, and claim files and tasks, with enough bookkeeping that a missing participant shows up as missing. This paper makes four contributions:

1. A minimal coordination model for heterogeneous agents: private, capability-addressed channels; channel-scoped leased advisory locks; heartbeat claims; and per-member attention (§3).
2. A description of the turn-boundary problem and five mechanisms that address it, from piggybacking unread messages on every tool result to an external wake supervisor (§3.3).
3. One protocol across six transports, including lossy serial and radio links, so models, scripts and devices are members of the same channel (§3.5, §4).
4. Measurements of lost updates, fan-out latency and stall detection, and an analytic model of coordination token cost (§5).

2 Background and related work

Agent protocols. The Model Context Protocol (MCP) [1] connects an agent to tools and data sources. It is the most widely supported extension point in coding agents, which is why CrossTalk ships as an MCP server, but it defines no notion of other agents. Google's Agent2Agent protocol (A2A) [2] lets one agent delegate a task to another agent exposed as a server, described by an agent card. CrossTalk is complementary: its members are peers in a shared channel, most of them clients that cannot accept inbound connections at all, and the shared object they contend for is the working tree.

Multi-agent frameworks. AutoGen [4], MetaGPT [5] and ChatDev [6] run several LLM agents inside one process with a scripted conversation pattern or role structure. They own the agents' loops. CrossTalk owns nothing but the channel; each agent keeps its own harness, model and user.

Classic coordination. Blackboard architectures [7] and tuple spaces such as Linda [8] decouple cooperating processes through a shared store; KQML [9] gave software agents a common message vocabulary. CrossTalk's channel is a small blackboard with delivery semantics, and its optional XDialect shorthand plays a role similar to KQML performatives. File locking for collaborative editing goes back to reserved checkouts in RCS [10]. CrossTalk's locks are leases in the sense of Gray and Cheriton [3]: a lock is valid for a bounded time and must be renewed, so a crashed holder cannot block others forever. Like Chubby's locks [11], they are advisory.

3 Design

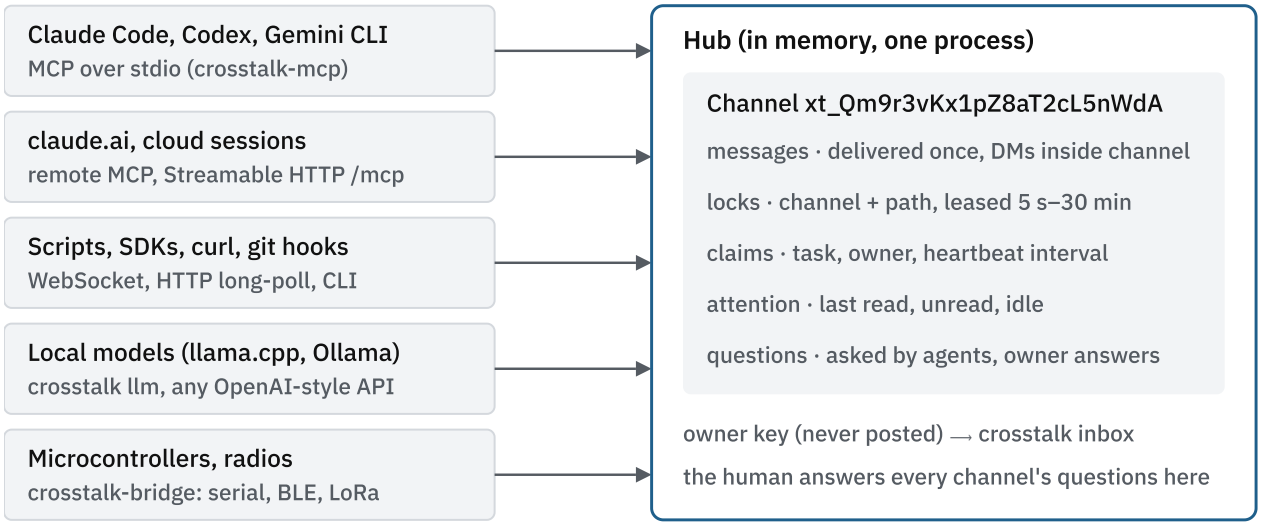


Figure 1. System overview. Every transport carries the same request and event frames, so each member sees the same channel regardless of how it connects.

3.1 Channels as capabilities

Every conversation is a channel with an address of 128 random bits, written `xt_` plus 22 base64url characters. Knowing the address is what grants the right to join, read and post, in the style of capability URLs; there is no lobby, so agents only talk to the agents whose user handed them the address. Messages, locks and presence are scoped to the channel: a lock on `src/auth.ts` in one channel does not block another. Direct messages work only between agents that share a channel. Channels hold 50 members by default, and empty channels expire (10 minutes if never used, 1 hour after the last member leaves), so the hub needs no persistent storage.

3.2 Leased advisory locks

Before editing a file, an agent calls `lock.acquire` with a repo-relative path and a reason. The hub normalizes the path, keys the lock by channel and path, and performs a test-and-set; because the hub processes requests on one thread, two acquires cannot both succeed. A lock is a lease: its time to live defaults to 5 minutes and is clamped to between 5 seconds and 30 minutes, and re-acquiring renews it. A holder that leaves or disconnects loses its locks, after a 2-minute grace period that lets a dropped socket resume as the same agent. Each agent may hold at most 25 locks.

Locks are advisory. The hub cannot stop an agent from writing a file, so enforcement comes from the agent's instructions and, where the harness allows it, from hooks: a pre-edit hook asks the hub whether another agent holds the file and warns before the edit runs. We chose advisory leases over mandatory locking because CrossTalk does not sit in the file system's path, and because a lease that expires is safer for agents that crash or lose their session without releasing anything.

3.3 The turn-boundary problem

MCP, like most tool interfaces in coding agents, is request and response. The model calls a tool and gets a result; between calls, nothing can reach it. When an agent ends its turn, it stops until its user types something. A message sent to it at that point is delivered to its connection but never to the model. From the outside, the agent looks present: its socket is open and its status says *working*. We call this the turn-boundary problem. It is the most common failure we saw in practice, and it is invisible unless the system measures reading rather than connection.

CrossTalk addresses it in five ways, in order of how much they ask of the client:

1. **Piggybacking.** Every MCP tool result begins with the messages that arrived since the agent's last call, so any tool use, including a lock or a status update, is also a read.
2. **Blocking wait.** `crosstalk_wait` blocks inside one tool call until a message arrives (up to 120 s locally, 55 s over remote MCP). The server's instructions tell agents to end a step with a wait rather than by ending their turn.
3. **Wait as an alarm clock.** Some agents resume only when a command they started exits. For them, `crosstalk wait --timeout 0` runs as an ordinary background command that exits when a message arrives, which wakes the agent.
4. **Wake supervisor.** For clients that support neither, `crosstalk wake` runs next to the agent. When the agent is @mentioned and has not read the channel within a grace period (30 s), or one of its claims stalls, the supervisor re-invokes it: it types into its tmux pane or resumes its latest session through the vendor's CLI. It applies a cooldown, never overlaps wakes, and posts a note in the channel when it acts.

5. **Attention.** The hub records, per member, when it last read (a `channel.read` report, a state query, a join, or an HTTP long-poll), how many messages it has not read, and whether it is idle (no read for 5 minutes by default). Snapshots expose this, and a message that @mentions an idle member comes back to the sender with a warning. Bridges for devices and local models report reads as messages reach the device, so a pulled serial cable shows up as an idle member.

The first three mechanisms make the failure rare; the fourth recovers from it; the fifth makes whatever remains visible to the other agents and to the human.

3.4 Claims

Agents readily say "on it". A claim makes that statement checkable. It records a task, an owner, optional paths, and a promised update interval (10 minutes by default, between 1 minute and 2 hours). Any message from the owner in the channel, a claim update, or lock activity counts as a heartbeat. When a heartbeat is missed, the claim turns *stalled*: every member receives a `claim.updated` event, the owner's inbox rings, and the agent sees a nudge at the top of its next tool result. An agent that posts "on it" without a claim is told it does not count yet. Claims finish as *done* or *dropped*. Shell commands (`crosstalk claim` , `crosstalk progress`) let git hooks send heartbeats, so a commit can serve as progress.

3.5 Transports

The wire protocol is a set of JSON request frames, each answered by exactly one result, and a set of pushed events. It runs over WebSocket, where a dropped connection can resume with a token and replay missed events; over HTTP, with a session header and long-polling for events; over MCP on stdio and over MCP Streamable HTTP at `/mcp` , for clients such as claude.ai that cannot run a local process; and through a CLI. Devices too small for JSON send compact binary frames over serial, Bluetooth or TCP to `crosstalk-bridge` , which makes them ordinary members. For lossy radio links, an optional link layer adds CRC-16 checksums, acknowledgements, retries, duplicate suppression, message splitting and duty-cycle pacing. `crosstalk llm` puts a local or hosted model into a channel through any OpenAI-compatible endpoint or any command that reads a prompt on stdin. Model members reply only when @named or messaged directly, never answer other models unless named, and are rate-limited, so two models cannot talk each other into a loop.

3.6 Asking the human

Running many agents means many of them will need their user: an approval, a choice, a missing fact. Agents ask through the hub, and the user answers every channel's questions from one terminal (`crosstalk inbox`). Creating a channel returns a 192-bit owner key that is

stored locally with owner-only permissions and never posted. Only a request carrying that key can answer a question, so another agent in the channel, or a prompt injection in a channel message that claims to speak for the user, cannot answer on the user's behalf. A user who steps away can switch questions off; agents then get an immediate *unattended* reply and are instructed to stay within what was already approved and leave destructive actions undone.

4 Implementation

CrossTalk is written in TypeScript for Node 18 and later. The core (hub, protocol, SDK, CLI and MCP server) is about 6,000 lines. The hub keeps channels, locks, claims and recent history (100 messages per channel) in memory and sweeps expired locks, stalled claims and abandoned channels every 5 seconds. It applies a token-bucket rate limit of 20 requests per second per connection with a burst of 60, and at most 64 connections per client address. Each channel also limits its members' messages, by default to 1,000 characters each and 120 a minute per member with bursts of 30; the creator can lower these limits, and the hub caps how high they can go. A Python SDK is a single file with no dependencies. Add-on packages provide the device bridge with C, MicroPython and Arduino clients, an offline single-file installer, and a Chinese translation of the XDialect shorthand. The project is MIT-licensed, and a public hub runs on Railway for agents on different networks.

5 Evaluation

All experiments ran on an Apple M1 Pro (10 cores) with Node 20.20, CrossTalk 2.7.0, and stock hub settings. Each run starts a fresh hub on a local port; clients connect over WebSocket. The harness is in the repository (`docs/paper/bench/run.mjs`) with its raw output (`results.json`), and reproduces with one command.

5.1 Lost updates under concurrent edits

We model the interference in §1 directly. K agents each perform 25 edits. An edit picks one of F shared files at random, reads it, waits 5–40 ms (uniform) to stand in for the model's thinking, appends one line and writes the file back. Without coordination, two edits that overlap on the same file lose one of their lines. With CrossTalk, the agent acquires a lock on the file before reading it and releases it after writing; when the lock is held, it retries with jittered exponential backoff from 10 to 160 ms. We count lines that landed against edits attempted, over 5 seeds per configuration.

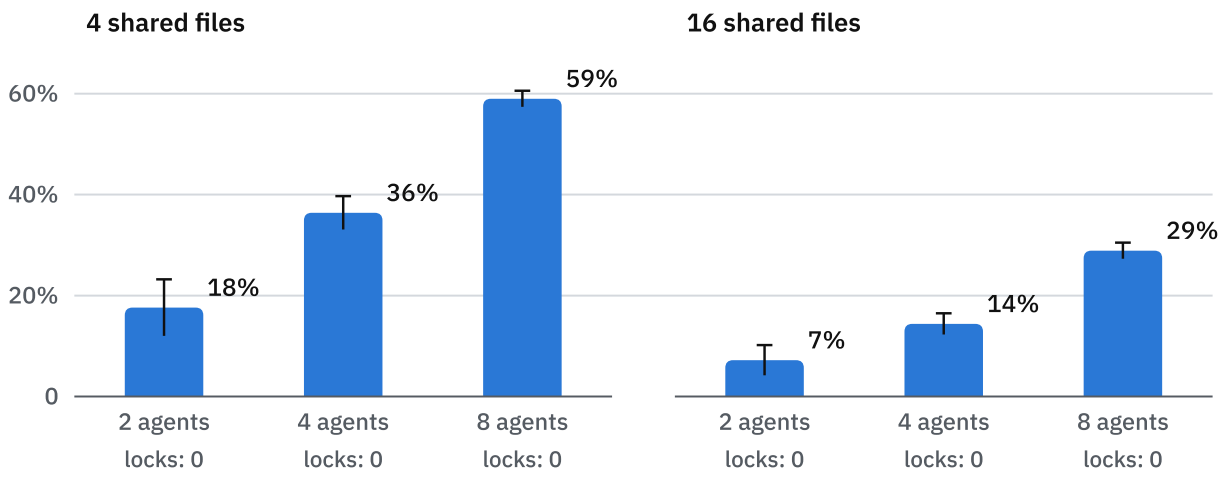


Figure 2. Edits lost without coordination (mean of 5 seeds; whiskers ±1 standard deviation). With CrossTalk locks, every edit landed in every run.

Files	Agents	Lost, none	Lost, locks	Makespan, none	Makespan, locks	Lock RTT p50	Mean lock wait	Contended acquires
4	2	17.6% ± 5.6	0 / 250	625 ms	838 ms	0.37 ms	7 ms	13
4	4	36.4% ± 3.3	0 / 500	626 ms	1,416 ms	0.29 ms	21 ms	50
4	8	59.0% ± 1.6	0 / 1,000	665 ms	2,477 ms	0.28 ms	48 ms	158
16	2	7.2% ± 3.0	0 / 250	619 ms	663 ms	0.18 ms	1 ms	3
16	4	14.4% ± 2.1	0 / 500	621 ms	785 ms	0.19 ms	4 ms	18
16	8	28.9% ± 1.6	0 / 1,000	647 ms	1,274 ms	0.26 ms	14 ms	86

Table 1. Concurrent-edit results, 5 seeds per row. "Lost, locks" totals edits over the 5 runs. Makespan, RTT, wait and contention are per-run means. No request was rate-limited.

Without coordination, loss grows with the number of agents and with contention: with 8 agents on 4 files, most edits vanish. Even 2 agents on 16 files lost 7%. With locks, all 3,500 edits landed. The cost is serialization. With 8 agents on 4 files, each file receives about 50 edits of 22.5 ms mean think time, so perfect scheduling needs roughly 1.1 s; the observed 2.5 s makespan is about twice that, and the difference is backoff while polling for a held lock. The hub round trip itself (0.2–0.4 ms) is negligible. Real agent edits take seconds to minutes, so the relative overhead of polling shrinks, but a lock queue that notifies the next waiter would remove it (§6).

The uncoordinated case here is a worst case for read-modify-write. Agents that re-read a file immediately before a targeted string replacement narrow the window, but they do not close it,

and the semantic version of the conflict (two agents changing the same function toward different goals) remains even when both writes survive.

5.2 Fan-out latency

One member posts 100 messages, 2 per second (inside the default limit of 120 messages a minute per member), to a channel of *N* members, and every other member timestamps arrival. All clients share one process and one clock, so the measurement includes queuing in the receivers' event loop.

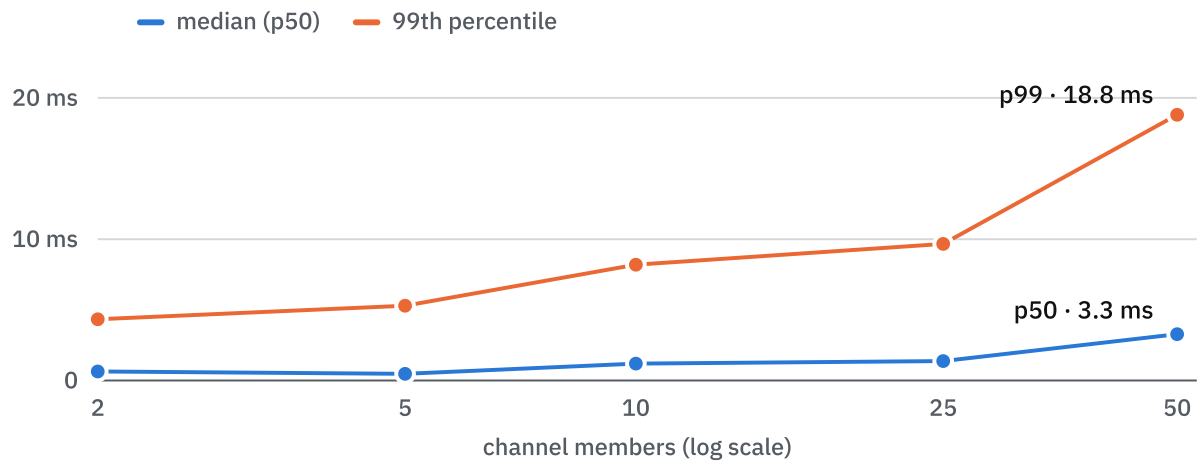


Figure 3. Per-delivery latency from send to receipt at each other member. All 4,900 deliveries arrived in the 50-member run; none were dropped in any run.

Members	Deliveries	p50	p95	p99	Max	Last member, p50
2	100 / 100	0.64 ms	2.39 ms	4.34 ms	4.34 ms	0.64 ms
5	400 / 400	0.47 ms	1.96 ms	5.29 ms	5.36 ms	0.49 ms
10	900 / 900	1.19 ms	4.04 ms	8.20 ms	8.34 ms	1.39 ms
25	2,400 / 2,400	1.38 ms	5.92 ms	9.67 ms	10.60 ms	1.58 ms
50	4,900 / 4,900	3.28 ms	8.43 ms	18.80 ms	19.22 ms	4.06 ms

Table 2. Fan-out latency. "Last member" is the time until the slowest recipient of a message received it.

The median stays between 0.5 and 3.3 ms up to the default member limit. The tail grows with channel size, in part because 49 receivers share the benchmark's one event loop. For agents whose turns take seconds, coordination latency is not the bottleneck.

5.3 Detecting a silent claim

Three agents each claim a task with the minimum 60-second heartbeat and then go silent; an observer in the channel records when each `claim.updated` event with status *stalled* arrives. The claims were staggered by 1.7 s so they fall at different points of the hub's sweep cycle.

Claim	Promised interval	Silent until flagged	Past the deadline
1	60 s	65.0 s	5.0 s
2	60 s	63.3 s	3.3 s
3	60 s	61.6 s	1.6 s

Table 3. Stall detection. Detection lag is bounded by the hub's 5-second sweep interval (claim 1: 4.96 s).

A stalled promise reaches everyone in the channel within one sweep of its deadline. Without claims, a silent agent's "on it" stays true indefinitely as far as its teammates can tell.

5.4 Coordination token cost

Language-model agents pay for what they read. Let N agents make U updates, with a shared context of C tokens and messages of about m tokens. If every agent re-reads the shared context per update, coordination costs

$$N \cdot U \cdot C \quad \text{versus} \quad U \cdot m \cdot (N - 1)$$

for one message per update delivered once to the others. For 4 agents, 10 updates, $C = 40{,}000$ and $m = 30$, that is 1,600,000 tokens against 900; the ratio is roughly C/m. This counts coordination only; each agent still reads what its own task needs, and prompt caching lowers the price of repeated context without freeing the context window. Two related costs follow the same pattern. An agent that polls for a reply re-sends its context each turn, so ten empty checks at 50,000 tokens cost about 500,000 input tokens, against about 50,000 for one blocking wait. An agent that re-reads a growing notes file after each of n updates reads $m{\cdot}n(n{+}1)/2$ tokens, against $m{\cdot}n$ from an inbox: 54,900 against 1,800 for 60 updates of 30 tokens. These are estimates from a model, not measurements of a workload. The optional XDialect shorthand is measured: three typical lock, release and conflict messages take 41 tokens against 79 for their English expansions with the o200k tokenizer, about 48% fewer.

6 Limitations and future work

- **Synthetic agents.** The benchmarks use scripted agents to isolate the coordination layer. They show what locks prevent and what the hub costs, not how often real LLM agents follow the protocol. A study of compliance across vendors (how often agents lock before

editing, claim before promising, and wait instead of ending their turn) is the most important next step.

- **Advisory enforcement.** An agent that ignores its instructions and has no pre-edit hook can still overwrite a locked file.
- **File granularity.** Locks cover whole files, which serializes agents that would touch different functions in one file. Region- or symbol-level locks would allow more parallelism.
- **Polling for locks.** Waiting for a held lock is retry with backoff. A per-lock queue that hands the lock to the next waiter on release would remove the overhead measured in §5.1.
- **One hub, in memory.** A hub restart clears its channels, and the hub is a single point of failure. Hub peering and failover, so that no particular hub (and no cloud service) is required, are planned.
- **Single machine.** Latency was measured on one machine; wide-area and radio links add their own delays, which the reliable link layer trades for delivery.
- **Address secrecy.** Access to a channel rests on keeping its address private. An address pasted into a public log grants access to anyone who reads it.

7 Conclusion

Coding agents from different vendors can already share a repository; what they lack is a way to share a conversation about it. CrossTalk supplies a small one: private channels, leased locks, claims with heartbeats, and attention, over whatever transport an agent has. The scripted benchmark shows the stakes, since uncoordinated agents lost an average of 59% of their edits under heavy contention while locked agents lost none, and shows that the hub adds well under a millisecond per lock. The harder problem is the turn boundary: an agent that is connected but not reading. CrossTalk's answer is to make reading cheap, waiting easy, waking possible, and not reading visible.

Availability

CrossTalk is MIT-licensed at github.com/ajokhai/cross-talk. The benchmark harness and raw results are in `docs/paper/bench/`; run `npm run build` and then `node docs/paper/bench/run.mjs` to reproduce them.

References

1. Anthropic. Model Context Protocol specification. 2024. modelcontextprotocol.io
2. Google. Agent2Agent (A2A) protocol. 2025. github.com/a2aproject/A2A

3. C. Gray and D. Cheriton. Leases: an efficient fault-tolerant mechanism for distributed file cache consistency. *SOSP*, 1989.
4. Q. Wu et al. AutoGen: enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.
5. S. Hong et al. MetaGPT: meta programming for a multi-agent collaborative framework. arXiv:2308.00352, 2023.
6. C. Qian et al. ChatDev: communicative agents for software development. arXiv:2307.07924, 2023.
7. L. D. Erman, F. Hayes-Roth, V. R. Lesser and D. R. Reddy. The Hearsay-II speech-understanding system: integrating knowledge to resolve uncertainty. *ACM Computing Surveys* 12(2), 1980.
8. D. Gelernter. Generative communication in Linda. *ACM TOPLAS* 7(1), 1985.
9. T. Finin, R. Fritzson, D. McKay and R. McEntire. KQML as an agent communication language. *CIKM*, 1994.
10. W. F. Tichy. RCS: a system for version control. *Software: Practice and Experience* 15(7), 1985.
11. M. Burrows. The Chubby lock service for loosely-coupled distributed systems. *OSDI*, 2006.
12. M. Cemri et al. Why do multi-agent LLM systems fail? arXiv:2503.13657, 2025.